

# 杂

皮克定理：顶点坐标均是整点的简单多边形，其面积  $S$  和内部格点数目  $a$  和边上格点数目  $b$  的关系为

$$S = a + \frac{b}{2} - 1$$

平面最近点对：如果平面上  $n$  个点的最近点对距离为  $d$ ，那么将这  $n$  个点放进  $d \times d$  的网格图中，每个网格中的点数不超过 4

```
1 // 平面最近点对, O(n logn)
2 T closest_pair(vector<Point> points) {
3     sort(points.begin(), points.end());
4     const auto cmpy = [](const Point &a, const Point &b) {
5         if(abs(a.y - b.y) <= eps) return a.x < b.x - eps;
6         return a.y < b.y - eps;
7     };
8     multiset<Point, decltype(cmpy)> s{cmpy};
9     T ans = inf;
10    for(size_t i = 0, l = 0; i < points.size(); i++) {
11        const T sqans = sqrtl(ans) + 1;
12        while(l < i && points[i].x - points[l].x >= sqans)
13            s.erase(s.find(points[l++]));
14        for(auto it = s.lower_bound(Point{-inf, points[i].y - sqans}); it !=
15            s.end() && it->y - points[i].y <= sqans; it++)
16            ans = min(ans, points[i].dis2(*it));
17        s.insert(points[i]);
18    }
19    return ans;
20 }
```

最小圆覆盖：如果点  $p$  不在点集  $S$  的最小覆盖圆内，那么它一定在  $\{p\} \cup S$  的最小覆盖圆上，即最小覆盖圆一定经过点  $p$

```
1 #ifndef usedouble
2 // 最小圆覆盖, 期望 O(n)
3 Circle min_enclosing_circle(vector<Point> p) {
4     if(p.empty()) return {{0, 0}, 0};
5     if(p.size() == 1) return {p[0], 0};
6
7     // 求三点外接圆
8     auto circumcircle = [](const Point &a, const Point &b, const Point &c) ->
9         Circle {
10         if(abs((b - a) ^ (c - a)) <= eps) {
11             T d1 = a.dis(b), d2 = a.dis(c), d3 = b.dis(c);
12             if(d1 >= d2 && d1 >= d3) return Circle{(a + b) / 2, d1 / 2};
13         }
14     };
15 }
```

```

12         if(d2 >= d1 && d2 >= d3) return Circle{(a + c) / 2, d2 / 2};
13         return Circle{(b + c) / 2, d3 / 2};
14     }
15     Point mab = (a + b) / 2;
16     Point vab = b - a;
17     Line l1 = {mab, {-vab.y, vab.x}}; // AB 的中垂线
18     Point mac = (a + c) / 2;
19     Point vac = c - a;
20     Line l2 = {mac, {-vac.y, vac.x}}; // AC 的中垂线
21     Point center = l1.inter(l2);
22     return Circle{center, center.dis(a)};
23 };
24
25 mt19937 rng(1919810);
26 shuffle(p.begin(), p.end(), rng);
27 Circle c = {p[0], 0};
28 for(size_t i = 1; i < p.size(); i++) {
29     if(c.is_in(p[i]) == 0) {
30         c = {p[i], 0};
31         for(size_t j = 0; j < i; j++) {
32             if(c.is_in(p[j]) == 0) {
33                 c = {(p[i] + p[j]) / 2, p[i].dis(p[j]) / 2};
34                 for(size_t k = 0; k < j; k++) {
35                     if (c.is_in(p[k]) == 0) {
36                         c = circumcircle(p[i], p[j], p[k]);
37                     }
38                 }
39             }
40         }
41     }
42 }
43 return c;
44 }
45 #endif

```